

Les systèmes de types

-

**Faire travailler l'ordinateur
pour vous, sans IA !**

Un type ? quésako ?

un **type** définit la nature des valeurs que peut prendre une donnée, ainsi que les opérateurs qui peuvent lui être appliqués. (Source : Wikipédia)

Quelques exemples :

```
a = 5  
b = "bonjour"  
c = []
```

Le typage dynamique : l'abstraction de Python

En Python, tous les types de variables sont **dynamiques** , ils n'ont pas besoin de rester statique pendant l'exécution d'un programme

Ce code est complètement valide :

```
a = 2352  
b = "bonjour"  
a = "au revoir"
```

C'est très pratique pour apprendre l'algo, mais c'est aussi à double tranchant !

Le typage dynamique : problèmes

Les types sont une information souvent extrêmement importante dans la compréhension et l'écriture de code

```
def write_to_db(data, database):  
    """Writes the given data to the database"""  
    ...
```

C'est quoi data ? un entier, une liste, un dictionnaire, une classe ?!

Et database ? une adresse IP, un chemin vers un fichier, une classe d'interface ?

Et la fonction, elle me renvoie quelque-chose ?

Le typage dynamique : solutions ?

Les indices de type et la documentation peuvent réduire le problème...

```
def write_to_db(data:dict, database:str) -> int:
    """
    Writes the given data to the database
    inputs :
        - data, a database dictionary update request, refer to documentation for
        details
        - database, URL to the database's API endpoint
    outputs :
        - int representing how many rows have been affected
    """
    ...
```

...mais est-ce qu'ils le règlent ?

Le typage dynamique : la gestion de l'erreur

```
def main():  
    # All application code, takes a LONG time, absolutely not scalable either  
    write_to_db([update1, update2], "127.0.0.1:2147") # please don't hack
```

L'erreur est **toujours** possible, et avec assez de temps et/ou de collaborateurs, **garantie**

Ici, si ce code arrive en production, on ne verra l'erreur que à la toute fin de l'exécution, j'espère que c'était pas des données importantes !

Le typage statique : une solution plus aboutie

Au lieu de laisser toutes les variables changer de type, et de laisser les types d'expressions être indéfinis/inconnus, tout est **statique**, connu **avant** l'exécution

```
def write_to_db(data:dict, database:str) -> int:  
    return 0
```

Ceci devient le détail minimum de la définition de notre fonction, et avant que l'exécution ne commence, un compilateur/interpréteur s'assurera que tous les appels sont valides !

Une erreur à l'exécution devient une erreur à la compilation, bien avant !

Le typage statique : une solution plus aboutie... vraiment ?

Le compilateur peut vérifier tous les types du monde... si y'a des failles dans le système de types, elles seront exploitées !

```
int magic_function(int a) {  
    return 1; // it's magic, don't question it  
}  
  
int main() {  
    int var1 = magic_function(42); // all good here  
    char *var2 = (char *)var1;    // this is where the fun begins  
  
    printf("var2 : %p\n", *var2); // instant UB  
  
    return 0;  
}
```

Le typage statique : pas suffisant

Dans beaucoup de langages typés statiquement, les types sont vérifiés, mais pas leur **validité** :

- Impossible d'empêcher la création d'une instance invalide d'un type
- Une fonction peut demander un type, mais est-ce qu'elle a besoin de garanties sur ce type ? (ex : pointeur spécifiquement non-null)
- Quand est-ce que les transformations de types sont possibles ou à éviter ?

=> Il devient difficile, voir **impossible** d'écrire un grand programme sans invalider un type à un moment donné, et la charge mentale est forte !

La “type safety” : comment transformer les types en contrats

Dans un langage “type safe”, la **validité** des types est vérifiée par le compilateur, exemple : la “référence” en Rust

- Doit pointer vers une adresse mémoire valide et actuellement allouée
- Ne permet pas de muter ce quoi vers elle pointe
- Ne peut être issue que d'un nombre fini d'opérations qui vérifient les invariants de son type*

Résultat : en Rust “safe” (le mode d'écriture par défaut du langage), **impossible** de créer un pointeur invalide

La “type safety” : l'exemple précédent

En reprenant l'exemple précédent en Rust, nous avons une erreur à la compilation :

```
fn magic_function(a:i32) -> i32 {  
    1 // it's magic, don't question it  
}  
  
fn main() {  
    let var1 = magic_function(42); // all good here  
    let var2:&[char] = var1.into();  
    // error[E0277]: the trait bound `&[char]: From<i32>` is not satisfied  
  
    println!("var2 : {}\n", var2);  
}
```

Il n'existe pas d'implémentation explicite de conversion entre i32 et &[char], donc la conversion est impossible

La “type safety” : les types d’expression

En étoffant la fonction de l’exemple, on peut voir une autre façon d’utiliser la type safety

```
fn magic_function (a:i32) -> i32 {  
    if a >= 42 {  
        println!("larger than or equal to 42" );  
        1  
        // it's magic, don't question it  
    }  
    else {  
        println!("smaller than 42" );  
    }  
}
```

Il y a une erreur, est-ce que vous la voyez ?

La “type safety” : les nouveaux paradigmes

Quand les types deviennent une forme de contractualisation du code, on peut faire travailler le compilateur !

Exemple : <https://doc.rust-lang.org/std/num/type.NonZeroU64.html>

Une famille de types numériques qui sont garantis de ne jamais valoir 0, le compilateur peut optimiser en conséquence

Les types génériques : donner plus de travail au compilateur

Les types peuvent prendre des paramètres, et ces paramètres peuvent être contraints :

```
2 implementations
pub trait World<ID:Identify>: Sized + Sync + Send + Clone {
    type WE:WorldEvent<Self, ID>;
    type RB:RenderingBackend;
    fn update_rendering(&mut self, data:&mut <Self::RB as RenderingBackend>::PreTickData);
}

2 implementations
pub trait WorldEvent<W:World<ID>, ID:Identify>: Sized + Sync + Send + Clone {
    fn apply_event(self, world:&mut W);
    fn get_source(&self) -> Option<ID>;
    fn should_sync(&self) -> bool;
}

#[derive(Clone)]
2 implementations
pub struct WorldHandler<W: World<ID>, ID:Identify> {
    pub world: Arc<RwLock<W>>,
    pub tunnels_in: WorldTunnelsIn<W, ID>,
    pub tunnels_out: WorldTunnelsOut<W, ID>,
    marker:PhantomData<ID>,
}
```

Les types génériques : Exprimer des contraintes

Exemple de mon moteur de rendu :

```
1 implementation
pub struct Vec3DfCam(pub Vec3Df);

impl Vec3DfCam {
    pub fn from_realspace(realspace:Vec3Df, campos:&Vec3Df, rotation:&Rotation) -> Self {
        Self(rotation.rotate(cible: realspace - campos))
    }
}

1 implementation
pub struct Vec3DfRaster(pub Vec3Df);

impl Vec3DfRaster {
    pub fn from_cameraspace(cameraspace:Vec3DfCam, viewport_data:&ViewportData) -> Self {
        let z: f32 = 1.0 / cameraspace.z;//(pos_finale.x.abs().clamp(0.001, INFINITY)).copysign(pos_finale.x);
        Vec3DfRaster(Vec3Df::new(
            x: (1.0 + (viewport_data.near_clipping_plane * cameraspace.x * z)) * viewport_data.half_image_width,
            y: (1.0 - (viewport_data.near_clipping_plane * cameraspace.y * z))
                * viewport_data.half_image_height
                * viewport_data.aspect_ratio,
            z,
        ))
    }
}
```

Les fonctions de rendu prennent que des Vec3DfRaster, il faut passer par la caméra pour les avoir !

Les types génériques : ça va très loin

Le système de types de Rust est Turing Complete, il est possible d'évaluer des conditions +/- arbitrairement compliquées sur les types

De l'arithmétique dans le système de types de Rust :

<https://fprasx.github.io/articles/type-system-arithmetic/>

Avec ces outils, vous pouvez rendre vos programmes plus robustes en exprimant leurs contraintes dans le système de types du langage

Fin

Fin...?

Les systèmes de type futurs : les constantes génériques

Aujourd'hui, on peut définir des types génériques sur des constantes :

```
pub struct Matrix<T:Nombre,const LENGTH:usize, const HEIGHT:usize> {  
    pub coord:[[T;HEIGHT];LENGTH]  
}
```

Mais on **ne peut pas** encore faire des opérations avec dans la définition de types :

```
pub struct Matrix<T:Nombre,const LENGTH:usize, const HEIGHT:usize> {  
    pub coord:[T; HEIGHT * LENGTH]  
}
```

Ça arrive ! la feature est en développement

Les systèmes de type futurs : les types dépendants

Les types dépendants sont un secteur de recherche actif dans la théorie de la programmation actuellement

En bref : des systèmes de types où il est possible d'exprimer des contraintes sur les **valeurs** d'expressions de différents types, sans avoir à les coder

Exemple : définition de types décimaux pour applications bancaires, ce serait moins cher si tout était prouvé par le compilateur !

Techniquement possible à différentes échelles dans tous les langages avec un système de types Turing Complete actuellement, mais souvent plus complexe à implémenter/écrire que l'alternative

Fin (pour de vrai)

<https://rust-lang.github.io/rust-project-goals/2026/flagships.html>

Crédits

Slides : Pierre Weisse